

CS23334 – FUNDAMENTALS OF DATA SCIENCE LAB

Experiment No. 1

Title:

Explore the fundamental concepts of data science through four short experiments: analyzing job posting trends, visualizing role distribution, differentiating data types, and encrypting/decrypting sensitive data.

Software Required:

Python 3, with the libraries pandas, matplotlib, seaborn, and cryptography.

Experiment 1.a — Trend of Data Science Job Postings

Aim:

To analyze the trend of data science job postings over the last decade using pandas for data manipulation and matplotlib for visualization.

Description

A small dataset of the number of data science job postings per year (2010–2020) is created and plotted as a line chart to observe the growth trend over time.

Sample Data Input

Year	= 2010, 2011, 2012, ..., 2020
Job Postings	= 150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200

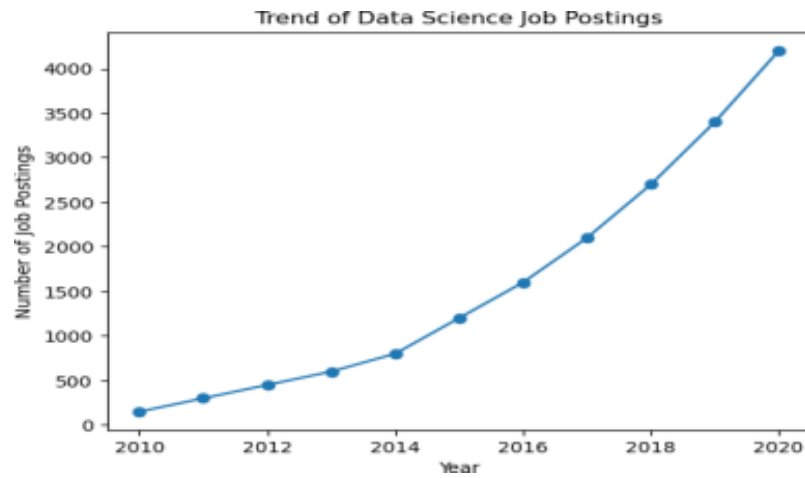
Program / Code

```
import pandas as pd
import matplotlib.pyplot as plt

data = {'Year': list(range(2010, 2021)),
        'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}
df = pd.DataFrame(data)

plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```

Output



Result

The line chart clearly shows a steady, accelerating rise in the number of data science job postings from 2010 to 2020, confirming the growing demand for data science roles over the decade.

Experiment 1.b — Distribution of Data Science Roles

Aim:

To analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, ML Engineer, Business Analyst) from a dataset using a bar chart.

Description

A dataset of job postings categorized by role is created, and the count of postings per role is visualized using a bar plot to compare demand across roles.

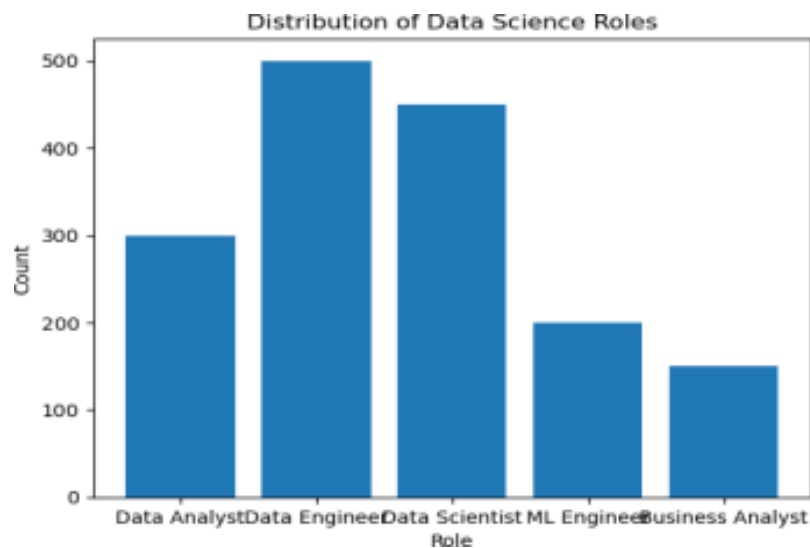
Sample Data Input

```
roles = Data Analyst, Data Engineer, Data Scientist, ML Engineer, Business Analyst  
counts = 300, 500, 450, 200, 150
```

Program / Code

```
import matplotlib.pyplot as plt  
  
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',  
         'Business Analyst']  
counts = [300, 500, 450, 200, 150]  
  
plt.bar(roles, counts)  
plt.title('Distribution of Data Science Roles')  
plt.xlabel('Role')  
plt.ylabel('Count')  
plt.show()
```

Output



Result

The bar chart shows that Data Engineer and Data Scientist postings are the most common roles, while Business Analyst postings are comparatively fewer, giving a clear picture of role-wise demand in the data science job market.

Experiment 1.c — Structured, Unstructured, and Semi-Structured Data

Aim:

To conduct an experiment differentiating structured, unstructured, and semi-structured data using sample datasets.

Description

Three small examples are created — a tabular pandas DataFrame (structured), a plain text string (unstructured), and a nested Python dictionary resembling JSON (semi-structured) — to illustrate the defining characteristics of each data type.

Program / Code

```
import pandas as pd

# Structured data example
structured_data = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})
print('Structured Data:\n', structured_data)

# Unstructured data example
unstructured_data = 'This is an example of unstructured data. It can be a piece of
text, an image, or a video file.'
print('\nUnstructured Data:\n', unstructured_data)

# Semi-structured data example (JSON)
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165,
'Weight': 68}}
print('\nSemi-structured Data:\n', semi_structured_data)
```

Output

```
Structured Data:
   ID  Name  Age
0   1  Alice  25
1   2   Bob  30
2   3 Charlie  35

Unstructured Data:
This is an example of unstructured data. It can be a piece of text, an image, or a
video file.

Semi-structured Data:
{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

Result

The experiment successfully demonstrates the three categories of data: structured data organized in a fixed tabular schema, unstructured data with no predefined format (free text), and semi-structured data that has some organization (key–value pairs) without a rigid schema.

Experiment 1.d — Encrypting and Decrypting Sensitive Data

Aim:

To encrypt and decrypt given sensitive data using the Fernet symmetric encryption scheme from the cryptography library.

Description

A secret key is generated using Fernet, which is then used to encrypt a piece of plain text into cipher text, and subsequently decrypt it back to verify the original message is recovered correctly.

Program / Code

```
from cryptography.fernet import Fernet

# Generate a key and create a cipher suite
key = Fernet.generate_key()
cipher_suite = Fernet(key)

# Encrypt the data
plain_text = b'Rajalakshmi Engineering College.'
cipher_text = cipher_suite.encrypt(plain_text)

# Decrypt the data
decrypted_text = cipher_suite.decrypt(cipher_text)

print('Original Data:', plain_text)
print('Encrypted Data:', cipher_text)
print('Decrypted Data:', decrypted_text)
```

Output

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABmmnHEhy1ZC1-Dox-
URNAvYqFFGfJemMb_fKmXqhATe3AAnXEOnSzaNRINI4
zAbNK8lgx2pprrNn-j6agbWdf30OwbrZsBuz8fnDdX7N35WwloDu3m6pVvkvw8GYDSI8Fov1tV'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

Result

The sensitive text was successfully encrypted into an unreadable cipher text using a Fernet key, and correctly decrypted back to the original message, verifying that the symmetric encryption and decryption process works as expected.